Data Abstraction, and Magic Method

🚨 Important Instruction for Week 07 Lab:

Do not use AI tools or external code generators to complete these exercises. They are intended for you to practice your own coding abilities. Think through the logic, reference Python syntax, and work through the challenges independently.

✏️ Your task is to write a Python Programming Language to complete the exercises below:

I. Data Abstraction

1. Airplane Ticket Booking System

Scenario:

Imagine you are designing a ticket booking system for an airline. There are different types of airplane tickets, such as *Economy Class*, *Business Class*, and *First Class*. Each ticket type has different pricing, services, and refund policies.

To ensure a consistent and structured approach, the airline hides the internal implementation of ticket processing and only allows customers to **book tickets**, **cancel bookings**, and **view ticket details** through a common interface.

Goal:

1. Create an abstract class `AirplaneTicket` that includes:
   - `__init__(self, passenger_name, ticket_price)`: Stores passenger details and ticket price.
   - `book_ticket(self)`: Marks the ticket as booked.
   - `cancel_ticket(self)`: Handles ticket cancellation with specific refund policies.
   - `display_ticket_info(self)`: Shows passenger details and ticket type.
   - Use @abstractmethod for `cancel_ticket()` and `display_ticket_info()` since each ticket type has different behavior.
2. Create three ticket types as subclasses:
   - **Economy Class Ticket** (Cheapest ticket with 50% refund if canceled.)

- **Business Class Ticket** (More expensive ticket with 70% refund if canceled.)
- **First Class Ticket** (Most expensive ticket with 90% refund if canceled.)

3. Simulate different cases by creating instances and testing bookings/cancellations.

Expected output:

```
Case Study 1: (Economy Ticket Booking and Cancellation)
    Ticket booked for Panha Reach - Price: $500
    Ticket canceled for Panha Reach. Refund Amount: $250.0
Case Study 2: (First Class Ticket Booking and Cancellation)
    Ticket booked for Alice Smith - Price: $2000
    Ticket canceled for Alice Smith. Refund Amount: $1800.0
```

Note: Think of other ticket types with special rules, like Student Discounts, Group Bookings, or Non-Refundable Tickets!

2. Library Management System

Scenario:

Imagine you are designing a Library Management System that allows members to borrow, return, and search for books. The system should handle different types of library items, such as **Books**, **Magazines**, and **Research Papers**, each with its own borrowing rules.

However, each type of item may have its own specific rules (e.g., a book can be borrowed for 14 days, while a research paper may have restricted access).

Suggestions:

1. Design an abstract class `LibraryItem` that provides a common structure for all items in the library.
2. Identify the important methods that should be included in `LibraryItem` but must be implemented differently by each specific item type.
3. Create at least two different item types (e.g., Book, Magazine, Research Paper) that inherit from `LibraryItem` and implement their own borrowing rules.

4. Add any additional functionality that makes the system more efficient and useful.

   For instance:

   - *How will you handle book availability?*
   - *Will there be penalties for late returns?*
   - *How can students and teachers have different borrowing privileges?*
   - *What happens if someone tries to borrow an unavailable item?*
   - *Can you add digital e-books that don't need physical returns?*

Expected output:

```
There is no fixed solution for this challenge. You are encouraged to think critically, design creatively, and implement your own unique Library Management System.
```

HINT: Think about how borrowing and returning should work. Should all items follow the same rules, or should different types have different conditions?

II. Magic Methods

3. Magic Methods in OOP

Tasks:

Imagine you are developing a system where different objects interact dynamically. Your task is to design a custom OOP model that involves at LEAST ONE magic method from each of the following categories:

- Initialization and Construction
- Numeric Magic Methods
- Arithmetic Operators
- String Magic Methods
- Comparison Magic Methods

Check out: *Dunder or magic methods in Python.* (2024, August 7). GeeksforGeeks. https://www.geeksforgeeks.org/dunder-magic-methods-python/

Idea: You are free to choose any real-world application that interests you, such as:

✔ A shopping cart system 🛒
✔ A virtual bank account 💰
✔ A video game character system 🎮
✔ A student grading system 🎓
✔ A book library system 📚
✔ A digital marketplace (buyers & sellers) 🏪
For example: A Shopping Cart (using `__init__`, `__add__`, `__str__`, `__eq__`)
- Items can be added dynamically to the cart.
- Two carts are equal if they have the same total price.

💡 *Note: Add an extra magic method to make your work even more interactive!*

***Reminder: Do not use AI tools** to generate your solution. Think through how you would logically check each condition and write your code accordingly.*

🖼️ At the end of this lab, you need to produce one file named: **full_name_W07_Lab.py**

💢 *Note: If you cannot submit the file in .py format, please submit it in a .zip or .txt format with the same filename (e.g., Chan_Dara_W07_Lab.zip).*